

Magic Squares

CS 221 Warmup Project

Objectives

- Read data from and write data to files
- Create and manipulate two-dimensional arrays
- Implement an interface
- Generate Java code from pseudocode

Background

For this project, you will write a program that checks whether a given ($n \times n$) matrix of integers values forms a magic square. Your program will also generate a magic square for a given n , assuming n is an odd number.

An ($n \times n$) matrix of integers filled with the values 1, 2, 3, ..., n^2 is a magic square if the sum of all of the elements in each row, in each column, and along the two diagonals is the same. For instance, the following is a 4×4 magic square with the magic sum of 34:

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

To determine the magic number of a given magic square, you can use this formula:

$$n(n^2 + 1) / 2$$

Tasks

You will write two classes for this project - a driver class and a class representing a potential magic square.

The first class you need to create is the MagicSquareDriver class. This is the driver class a user runs to create a MagicSquare object and get output from its operations. It should include your main method in which command-line arguments are read and validated and a MagicSquare is read from file or created.

1. To operate correctly, MagicSquareDriver should run from the command line as follows:



```
java MagicSquareDriver <-check | -create> <filename> < |size>
```

where either a -check or -create flag and a filename is required. The size should only be included when the create flag is used and must be an odd positive integer.

2. The -check flag indicates that your program will determine whether the file called filename contains a magic square and will print out the appropriate message.

For instance, to determine whether a file called myMatrix.txt contains a valid magic square, you would use the following command-line syntax:

```
java MagicSquareDriver -check myMatrix.txt
```

and, assuming it contains the 4x4 magic square shown above, the following message should print out on your console:

```
The matrix
    16 3 2 13
    5 10 11 8
    9 6 7 12
    4 15 14 1
is a magic square.
```

If the matrix in the file is not a magic square, your program should print out the same output, except it should state that the matrix "is not a magic square."

3. With the -create flag, your program will create a magic square with the dimensions size x size and will write that square to the file called filename. Also print the matrix to the console in the same format as shown for the check flag.

For instance, to create a 5 x 5 magic square that is written to the file called myMatrix.txt, you would use the following command-line syntax:

```
java MagicSquareDriver -create myMatrix.txt 5
```

and, since the size was odd, your program would write the magic square to the given file in the same file format as shown in sample input files. Your program should also print the matrix and the result of confirming that it is a magic square to the console. If the size was not odd, no file should be written and a usage message should be printed to the console specifying the requirement that the size be odd.

4. Be sure to validate the command-line arguments. If the arguments are in the wrong order, if there are not the expected number of arguments, or if the size argument is not odd, print a clear [usage message](#) and exit the program.



Create a class called MagicSquare that implements the [MagicSquareInterface](#).

1. Required constructors and methods (including required private methods) for MagicSquare are described in the interface. Read all documentation in the interface carefully and thoroughly to understand what is expected from the class and each of the specified interface methods.
2. The only two allowed instance variables in MagicSquare are a 2D int array for the matrix and an optional boolean for whether it is a valid magic square.
3. You are expected to follow Java naming conventions for all class, method, and variable names.
4. You are expected to enforce object encapsulation throughout all code.
5. There should be no console output apart from what is described in this document.
6. Match file and console output formats to formats shown in this document.
7. Files this program reads and writes have a specific format: The size of the matrix appears in the first line of the file. The rest of the file should contain the values of the matrix using that number of rows and columns. For instance, a 5 x 5 magic square could be stored in a file as:

```
5
11 18 25 2 9
10 12 19 21 3
4 6 13 20 22
23 5 7 14 16
17 24 1 8 15
```

8. To read/parse a file, you should use the Scanner class, which reads basic data from files. You are advised to use one Scanner to read the file a line at a time, and additional Scanners to parse each line.
9. To determine whether a matrix is a magic square, you have to ask these questions:
 - a. Do each of the integers 1, 2, 3, ..., n^2 appear in the matrix?
 - b. Are the sums of each of the rows, columns, and the two diagonals the same value, which is equal to the magic number $(n(n^2 + 1) / 2)$?
10. To create a magic square, use this algorithm:

```
Create an integer value n = size of the magic square.
Create a two-dimensional array of size n.
Create two integer values, row and col.
Set row = n - 1.
Set col = n / 2.
Create two integer values, old-row and old-col.
Loop through values from i = 1 to i = n2
    Place the value of i in the array at the location [row][col].
```



```

Set old-row = row and old-col = col.
Increment row and col.
If row == n, then replace it with 0.
If col == n, then replace it with 0.
Check the value stored at the location [row][col].
If the element at [row][col] has already been filled,
    Set row = old-row,
    Set col = old-col, and
    Decrement row.

```

Here is the 3 x 3 magic square created using this method:

4	9	2
3	5	7
8	1	6

11. To open a file stream to which you can write, use the following code:

```

File file = new File(filename);
PrintWriter outFile = new PrintWriter(file);
...
outFile.close();

```

where filename is a String variable with the name of the file you're writing to. Note that this code will overwrite the contents of the file if it already exists. The missing code should be filled in with the appropriate method calls on the PrintWriter variable.

The [MagicSquareTest](#) test driver class is given to help confirm the correct implementation of MagicSquare. This [zip file](#) includes input files for both valid and invalid matrices needed by the test class and that you can use for ad hoc testing of MagicSquareDriver.

Documentation and Code Quality

Write a plain-text README file, called README, with [this format and content](#). (For reference, here is a [sample README file](#) from another project.)

Include [Javadoc comments](#) for all classes and methods. You are expected to produce well-documented code with meaningful identifier and method names that follow standard Java naming conventions.

All identifier and method names are expected to be meaningful and follow standard Java naming conventions. All code is expected to be consistently and appropriately indented and have consistent and appropriate vertical spacing between blocks of code. See [CS221 Style Guide](#) for more.



Only appropriate and necessary object state data should be declared as instance variables. All other variables should be declared locally in methods. Use method parameters and return values to pass non-instance data between methods.

Grading

Points will be awarded according to the following breakdown:

Tasks	Points
Documentation: README file, javadoc comments	10
MagicSquareDriver command line handling	5
Output / toString()	5
MagicSquare functionality (tester results)	25
Code Quality - formatting, naming conventions, encapsulation, etc.	5

Required Files

Submit the following files:

- README
- MagicSquareDriver.java
- MagicSquare.java
- MagicSquareInterface.java
- *Any additional .java files necessary to compile this project.*

Submission

Place all assignment files into a directory in your onyx account. Do not include any unnecessary files. Use the submission command given on your section's class web page from the directory containing your files on onyx.

